

1. Introducción y Objetivos

El presente informe documenta el procedimiento técnico llevado a cabo para la habilitación, registro y personalización del panel de administración provisto por el framework Django en el proyecto del sistema de arriendo de inmuebles. El objetivo principal consiste en proveer a los administradores de una interfaz visual eficiente para la gestión de entidades territoriales (**Region y Comuna**) y propiedades habitacionales (**Inmueble**), garantizando herramientas de filtrado y búsqueda avanzada.

2. Requerimiento 1: Creación del Superusuario

Para contar con credenciales de acceso con privilegios totales sobre el panel de administración, se ejecutó en la terminal integrada el comando nativo de gestión de Django: `python manage.py createsuperuser`

Parámetros configurados:

Username: `admin`

Email address: `conymenaba@gmail.com`

Password: Configurada de forma segura en el entorno local.

El sistema confirmó la persistencia del usuario en la tabla `auth_user` de la base de datos PostgreSQL con el mensaje `Superuser created successfully`.

3. Requerimiento 2: Definición y Registro de Modelos en el Admin

En el archivo `inmuebles_app/models.py` se definieron las entidades geográficas requeridas (**Region y Comuna**), asociando esta última mediante una clave foránea (**ForeignKey**) con eliminación en cascada (`models.CASCADE`). Asimismo, se vinculó cada **Inmueble** con su respectiva comuna.

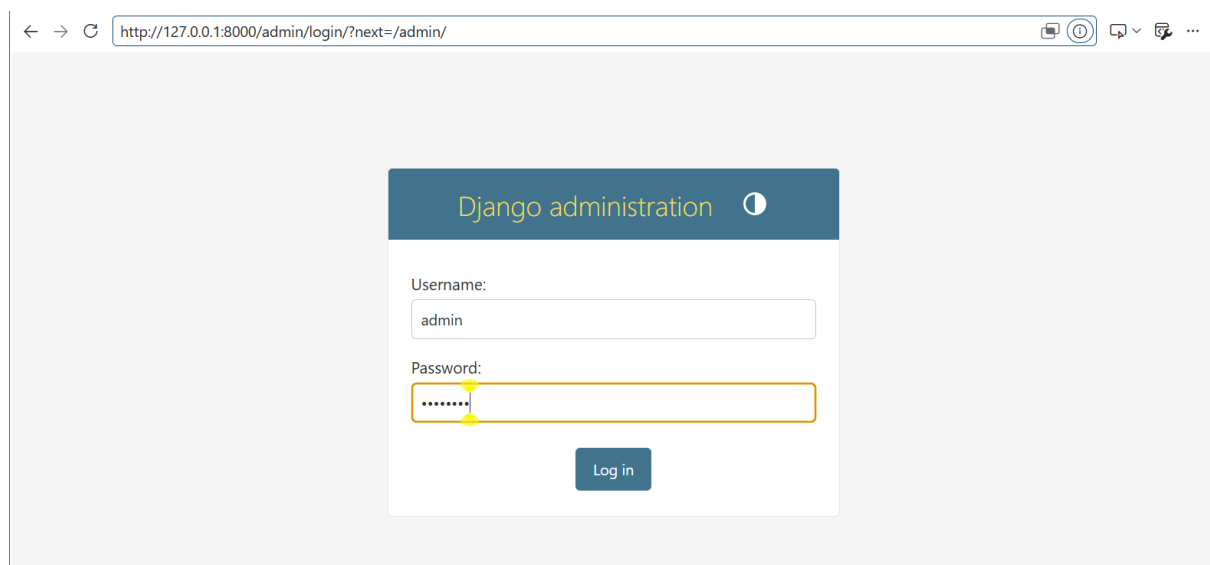
Para exponer dichos modelos en el panel, se implementó el archivo `inmuebles_app/admin.py` utilizando los decoradores `@admin.register()`, los cuales ejecutan internamente la vinculación equivalente a `admin.site.register()`:

```
from django.contrib import admin
from .models import Inmueble, Region, Comuna
```

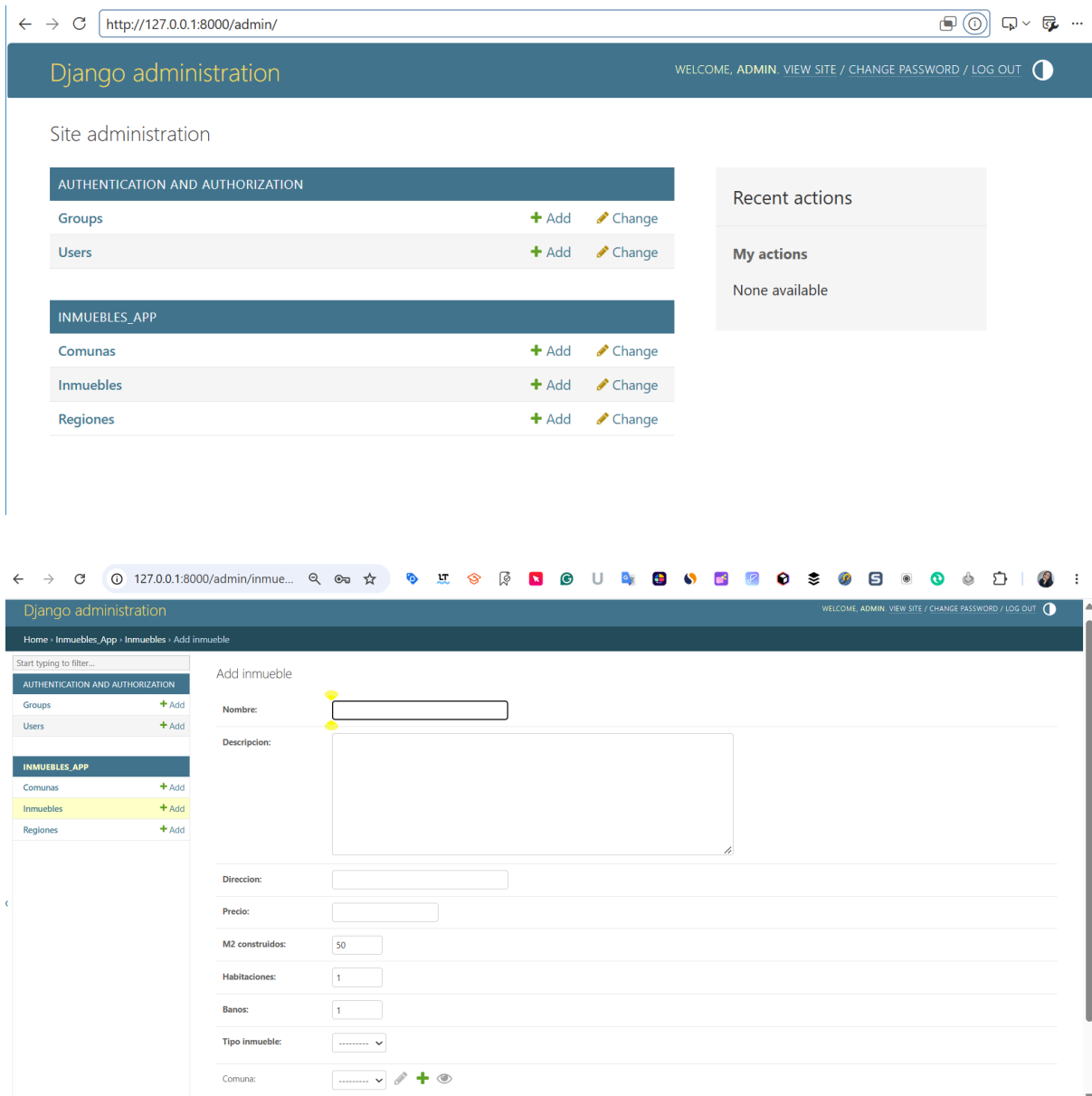
```
@admin.register(Region)
class RegionAdmin(admin.ModelAdmin):
    list_display = ('id', 'nombre')
    search_fields = ('nombre',)
```

```
@admin.register(Comuna)
class ComunaAdmin(admin.ModelAdmin):
    list_display = ('id', 'nombre', 'region')
    list_filter = ('region',)
    search_fields = ('nombre', 'region__nombre')
```

Al iniciar el servidor (`python manage.py runserver`) e ingresar a la URL `http://127.0.0.1:8000/admin/`, se presenta la pantalla de autenticación protegida.



Tras iniciar sesión con las credenciales creadas, se visualiza el menú principal donde se disponen ordenadamente las entidades **Comunas**, **Inmuebles** y **Regiones** bajo la aplicación `Inmuebles_App`.



4. Requerimiento 3: Personalización del Panel de Administración

Para optimizar la usabilidad del panel, se configuró la clase `InmuebleAdmin` en `admin.py` incorporando las tres directivas obligatorias de visualización:

- **list_display:** Define las columnas visibles en la tabla principal (`id`, `nombre`, `tipo_inmueble`, `comuna`, `precio`).
- **list_filter:** Habilita un panel lateral derecho para segmentar los inmuebles por categoría (`tipo_inmueble`), región (`comuna__region`) y comuna específica.
- **search_fields:** Añade una barra superior de búsqueda que permite realizar consultas en tiempo real por el nombre del inmueble, dirección y descripción.

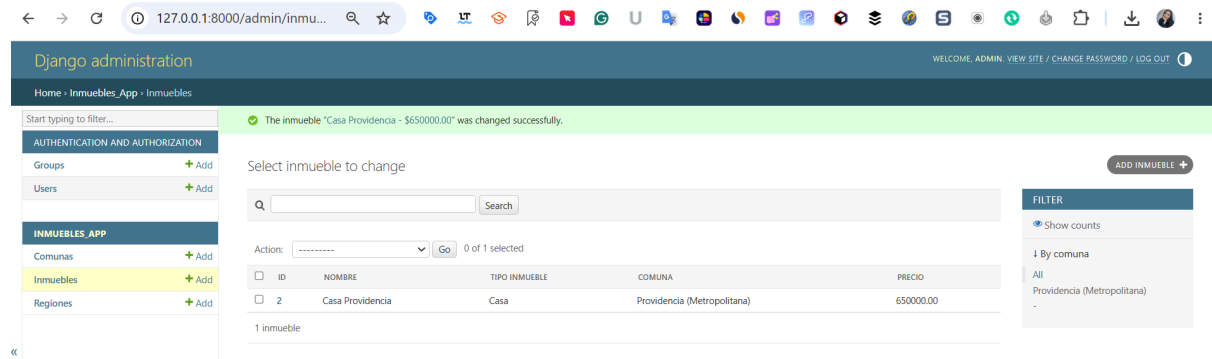
```
@admin.register(Inmueble)
```

```
class InmuebleAdmin(admin.ModelAdmin):
```

```
    list_display = ('id', 'nombre', 'tipo_inmueble', 'comuna', 'precio')
```

```
list_filter = ('tipo_inmueble', 'comuna__region', 'comuna')
search_fields = ('nombre', 'direccion', 'descripcion')
```

Esta configuración permite al operador encontrar registros de forma inmediata y filtrar el catálogo sin requerir consultas SQL manuales.



5. Conclusión

El panel de administración quedó plenamente operativo bajo Django 5.2 conectado a PostgreSQL. Se satisfacen cabalmente los criterios de la escala de apreciación: creación de superusuario, registro de los tres modelos de negocio requeridos y personalización orientada a la experiencia de usuario (`list_display`, `search_fields`, `list_filter`).